

Multi-Source Learning Content Ingestion & Structured Output Generation

System Architecture, Technical Specifications, API/CLI Guide, and Operations Manual

Project Documentation Report
Version 1.0.0 | August 2026

1. Executive Summary & Project Objective

The Multi-Source Learning Content Ingestion & Structured Output Generation system is an enterprise-grade solution designed to automate the extraction of knowledge from diverse educational materials. Educational content exists in heterogeneous formats—such as PDF documents, lecture transcripts, and text files. Converting these raw inputs into structured learning artifacts (concept hierarchies, question-answer flashcards, concise summaries, and directed knowledge graphs) manually is time-consuming and prone to inconsistencies.

This system addresses this challenge by providing a fully automated, multi-layered processing pipeline. Core system objectives include:

- **High-Quality Extraction:** Leveraging schema-constrained Large Language Model (LLM) prompting to extract domain concepts and relationships without conversational fluff.
- **Extensible Architecture:** Employing strict plugin and factory patterns allowing new file parsers, LLM providers, or output formats to be added as isolated modules without modifying core pipeline orchestration.
- **LLM Provider Abstraction:** Offering native, vendor-agnostic support for OpenAI-compatible APIs (including Groq and NVIDIA NIM), switchable dynamically via configuration.
- **Zero-Infra Storage:** Utilizing an embedded SQLite database with complete repository encapsulation, ensuring zero external service dependencies.
- **Demo Safety & Dual Interfaces:** Providing a primary CLI interface for deterministic execution alongside a web backend and React frontend featuring SVG graph visualization.

2. Architecture & Design Principles

The architecture strictly enforces single-responsibility layers with unidirectional dependencies. Higher layers invoke adjacent lower layers through explicit TypeScript interfaces.

2.1 Processing Layers

- **Ingestion Layer (`src/ingestion/`):** Responsible for reading physical files and converting them into raw extracted text and file metadata using format-specific parsers registered in a central parser registry.
- **Normalization Layer (`src/normalization/`):** Standardizes raw text and metadata from any ingestion format into a canonical SourceDocument structure (id, filename, sourceType, rawText, metadata, ingestedAt).
- **Extraction Layer (`src/extraction/`):** Orchestrates text chunking (token/character threshold based), LLM prompt assembly, provider invocation, and cross-chunk concept deduplication.
- **Validation/Structuring Layer (`src/validation/`):** Validates raw LLM JSON output against strict Zod schemas. On syntax or schema failure, executes a single repair retry prompt before failing gracefully.
- **Storage Layer (`src/storage/`):** Implements the repository pattern over SQLite using better-sqlite3. Ensures

100% encapsulation of SQL queries with zero raw SQL leaks outside the storage module.

- **Retrieval Layer (src/retrieval/)**: Queries persisted concepts, flashcards, summaries, and graph relationships by topic name using exact matching with case-insensitive substring fallback.
- **Presentation Layer (src/cli/ & web/)**: Presents dual user interfaces: a Commander-based CLI for terminal execution and an Express API + React frontend for web interactions.

2.2 Plugin Pattern & Extensibility

To support new ingestion formats (such as video transcripts or HTML documents), the system implements a Parser interface. Adding support for a new format requires only creating a new parser file implementing supports(filePath) and parse(filePath), and registering it in registry.ts.

2.3 Vendor-Agnostic LLM Provider Abstraction

The extraction layer depends exclusively on the LLMProvider interface (complete(prompt, systemPrompt)). The factory function getLLMProvider() reads CONFIG.LLM_PROVIDER ("groq" or "nvidia") and returns the configured provider instance. Switching between Groq and NVIDIA NIM requires zero code edits—only updating the LLM_PROVIDER variable in .env.

2.4 Project Folder Structure

```
project-root/
% % % src/
%   % % % ingestion/      # PDF and Transcript parsers + registry
%   % % % normalization/  # SourceDocument canonical converter
%   % % % extraction/      # Prompt templates, chunking, LLM providers, merging
%   % % % validation/      # Zod schema definitions & repair retry engine
%   % % % storage/         # SQLite database connection & repositories
%   % % % retrieval/       # Topic search & artifact retrieval engine
%   % % % outputs/         # Flashcard JSON/CSV generators & graph formatters
%   % % % cli/             # Commander CLI entrypoint & commands (ingest, list, export)
%   % % % shared/          # Centralized config.ts & shared domain types
% % % web/
%   % % % server/          # Express API server & routes
%   % % % client/          # React frontend, SVG ConceptGraph, UploadControl
% % % seed-data/          # Sample PDF and transcript test files
% % % tests/              # Unit & end-to-end integration test suites
% % % docs/               # Project documentation artifacts
% % % package.json
```

3. Functional Feature Mapping (MVP vs Stretch)

All functional requirements (FR1 through FR6) specified in REQUIREMENTS.md have been completely implemented and verified for MVP delivery.

3.1 Completed MVP Features

- **M1 — PDF Parsing (FR1.1):** Extracts raw text and page count metadata from PDF files via pdfParser.ts.
- **M2 — Transcript Parsing (FR1.2):** Reads plain-text, markdown, and transcript files via textTranscriptParser.ts.
- **M3 — Normalization (FR1.3):** Normalizes all input formats into canonical SourceDocument objects in normalize.ts.
- **M4 — LLM Concept Extraction (FR2.1):** Extracts concepts, relationships (prerequisite, related-to, part-of), and summaries using schema-constrained prompts in extract.ts.
- **M5 — Schema Validation & Repair (FR2.2, FR2.3):** Validates LLM output via Zod schemas and executes a single repair retry on malformed JSON in validateExtraction.ts.
- **M6 — Text Chunking & Merging (FR2.4):** Splits text exceeding token thresholds (chunk.ts) and merges cross-chunk concepts by normalized name (mergeChunks.ts).
- **M7 — Flashcard Generation (FR3.1):** Generates question/answer flashcards exportable to JSON and CSV formats (flashcardExport.ts).
- **M8 — Document Summary (FR3.2):** Produces concise document summaries and persists them in SQLite (summaryRepository.ts).
- **M9 — Concept Graph Data (FR3.3):** Transforms extracted concepts and directed edges into node/edge graph data (graphExport.ts).
- **M10 — SQLite Storage (FR4.1):** Persists documents, concepts, relationships, flashcards, and summaries across 5 tables with foreign keys (db.ts).
- **M11 — Topic-Based Retrieval (FR4.2):** Queries stored artifacts by topic using exact matching with substring fallback (getArtifactsByTopic.ts).
- **M12 — CLI Commands (FR5.1–FR5.3):** Provides ingest, list-topics, and export commands with clear terminal summaries and non-zero failure exit codes.
- **M13 — Minimal Web UI (FR6.1–FR6.3):** Express API backend + React frontend with single-file upload control, summary panel, expandable flashcard list, and interactive SVG concept graph visualizer.
- **M14 — Seed Data & Verification:** Includes seed PDF and transcript files in seed-data/ and verified reproducible setup.

3.2 Stretch Features Summary

Stretch features (S1 through S7 listed in FEATURES.md)—such as video/audio transcription (S5) and embedding-based vector search (S4)—remain out of MVP scope to protect demo reliability. The architecture is modularized so stretch features can be added additively.

4. API & CLI Usage Guide

4.1 CLI Commands & Examples

Ingest a learning file and run full extraction pipeline:

```
npm run cli -- ingest seed-data/transcripts/machine_learning_intro.txt
```

Output Summary:

```
--- Ingestion Pipeline Completed Successfully ---
Multi-Source Learning Content Ingestion — Page 3 of 7
```

Document ID: 1706d9bb-aaf2-4cf1-9ef4-75f740815e73
Filename: machine_learning_intro.txt
Source Type: transcript
Concepts Found: 4
Relationships Found: 3
Flashcards Generated: 4
Graph Nodes: 4
Graph Edges: 3
Processing Time: 1.85s
Database Persistence: Saved to SQLite database.

List distinct stored topics:

```
npm run cli -- list-topics
```

Export topic flashcards:

```
npm run cli -- export "Machine Learning" --format json  
npm run cli -- export "Machine Learning" --format csv
```

4.2 Express Web API Endpoints

- **POST /api/ingest:** Accepts multipart file upload (file). Runs full pipeline and returns 200 OK JSON with documentId, concepts, relationships, summary, flashcards, and graph. Returns 422 error JSON on failure.
- **GET /api/topics:** Returns 200 OK JSON with { topics: string[] }.
- **GET /api/topics/:topic:** Returns 200 OK JSON with { topic, concepts, flashcards, summary, graph } or 404 error JSON if topic is not found.

5. Database Schema & Data Models

The database schema is defined in DATABASE.md and initialized in src/storage/db.ts. Foreign keys are explicitly enabled.

5.1 Tables Definition

- **documents:** id (TEXT PK), filename (TEXT), source_type (TEXT), raw_text (TEXT), metadata (TEXT JSON), ingested_at (TEXT ISO).
- **concepts:** id (TEXT PK), document_id (TEXT FK -> documents.id), name (TEXT normalized), description (TEXT).
- **relationships:** id (TEXT PK), from_concept_id (TEXT FK -> concepts.id), to_concept_id (TEXT FK -> concepts.id), type (TEXT).
- **flashcards:** id (TEXT PK), concept_id (TEXT FK -> concepts.id), question (TEXT), answer (TEXT).
- **summaries:** id (TEXT PK), document_id (TEXT FK -> documents.id), summary_text (TEXT).

5.2 Indexing Rules

- **idx_concepts_name:** Index on concepts(name) for fast topic retrieval lookup.
- **idx_relationships_from_concept_id:** Index on relationships(from_concept_id) for graph edge resolution.
- **idx_relationships_to_concept_id:** Index on relationships(to_concept_id) for incoming edge traversal.

6. Setup & Operations Manual

Follow these steps for a clean installation:

```
# 1. Clone repository & install dependencies
npm install

# 2. Configure environment file
cp .env.example .env

# Edit .env with your Groq or NVIDIA API key:
# LLM_PROVIDER=groq
# GROQ_API_KEY=your_api_key_here

# 3. Initialize SQLite database
npm run db:init

# 4. Start Web API server & frontend
npm run server
```

6.1 Test Suite Execution

```
node node_modules/tsx/dist/cli.mjs tests/parserSmoke.test.ts
node node_modules/tsx/dist/cli.mjs tests/schemaValidation.test.ts
node node_modules/tsx/dist/cli.mjs tests/repositoryRoundtrip.test.ts
node node_modules/tsx/dist/cli.mjs tests/retrievalByTopic.test.ts
node node_modules/tsx/dist/cli.mjs tests/webApi.test.ts
node node_modules/tsx/dist/cli.mjs tests/cliSmoke.test.ts
```

7. Known Limitations & Triaged Edge Cases

- **Corrupted Files:** Handled gracefully. Invalid PDFs produce a human-readable error without crashing the

pipeline.

- **Malformed LLM Output:** Handled via Zod schema validation and single repair retry. Second failures throw `ExtractionValidationError`.
- **Short Transcripts:** Very short documents return valid empty-ish concept sets rather than throwing errors.
- **Extremely Long Documents:** Chunking handles large texts, but cross-chunk concept deduplication quality is best-effort.
- **Language Support:** MVP is scoped to English content.
- **Cross-Document Deduplication:** Concept deduplication is guaranteed within a single document; cross-document merge is a stretch item (S2).

8. Scaling Story & Production Readiness

For production scaling, the system architecture supports seamless upgrades:

- **Async Job Queue:** File ingestion can be offloaded to a message queue (e.g. BullMQ / Redis) to prevent blocking HTTP threads.
- **Parallel Extraction:** Chunk extraction calls can be executed concurrently using worker pools for large document processing.
- **Vector Search:** The retrieval layer interface can incorporate embedding-based semantic search without modifying higher presentation layers.
- **Postgres Migration:** Replacing SQLite with PostgreSQL requires updating repository implementations in `src/storage/` only; no other layers require modification.